

ProyectoIntegrador Fintech etapa 2

Me parece una mejora importante, sobre todo porque tiene un enfoque más cercano a una aplicación real y es más útil para los alumnos. Yo haría dos cambios en lugar de uno.

1. Reingresar el dato cuando es inválido

En lugar de terminar la aplicación con el primer error, cada dato se solicita hasta que sea válido.

Por ejemplo, para el ID:

JavaScript

```
let id;

while (true) {

    try {

        id = Number(prompt("Ingrese el ID del cliente:"));

        // Aprovechamos las validaciones de la clase Cliente
        const clienteTemporal = new Cliente(
            id,
            "Temporal",
            "Temporal",
            "12345678",
            "temp@gmail.com",
            "password123"
        );

        break;

    }
    catch(error){

        alert(error.message);

    }

}
```

Sin embargo, existe una solución mejor.

En vez de crear objetos temporales, podemos hacer que la clase **App** tenga un método reutilizable como éste:

JavaScript

```
pedirDato(mensaje, setter) {  
  
    while (true) {  
  
        try {  
  
            const valor = prompt(mensaje);  
  
            setter(valor);  
  
            return;  
  
        }  
        catch(error){  
  
            alert(error.message);  
  
        }  
  
    }  
  
}
```

Luego, durante la creación del cliente:

JavaScript

```
const cliente = new Cliente(  
    1,  
    "Temporal",  
    "Temporal",  
    "12345678",  
    "temp@gmail.com",  
    "password123"  
);  
  
this.pedirDato(  
    "Ingrese el nombre:",  
    valor => cliente.setNombre(valor)  
);
```

Así reutilizamos directamente todas las validaciones de la clase `Cliente` sin escribirlas nuevamente en `App`.

2. Mostrar todo en pantalla

En lugar de depender de la consola (`F12`), agregaría una especie de "terminal" dentro de la página.

Por ejemplo, en el HTML:

HTML

```
<div id="salida"></div>
```

y un estilo similar a una consola:

CSS

```
#salida{  
  
    margin-top:25px;  
  
    background:#263238;  
  
    color:#00FF66;  
  
    font-family:Consolas, monospace;  
  
    padding:20px;  
  
    border-radius:8px;  
  
    min-height:250px;  
  
    white-space:pre-line;  
  
    overflow-y:auto;  
  
}
```

Entonces, desde `App.js`, en lugar de hacer únicamente:

JavaScript

```
console.log("Cliente creado.");
```

hacemos:

JavaScript

```
mostrar(mensaje){  
  
    console.log(mensaje);  
  
    document.getElementById("salida").innerHTML +=  
        mensaje + "<br>";  
  
}
```

y todo aparecerá simultáneamente:

- en la consola (para quien quiera verla),
- y en la propia página.

3. Incluso podríamos mostrar el estado del objeto

Cada vez que el usuario modifica un dato, podríamos actualizar un panel como éste:

```
=====
          CLIENTE ACTUAL
=====

ID.....: 1

Nombre.....: Juan

Apellido.....: Pérez

DNI.....: 32145678

Email.....: juan@gmail.com

=====
```

Después de cambiar el apellido:

```
=====
          CLIENTE ACTUAL
=====

ID.....: 1

Nombre.....: Juan

Apellido.....: González

DNI.....: 32145678

Email.....: juan@gmail.com

=====
```

Eso resulta muy visual para explicar el concepto de **estado de un objeto**, ya que los alumnos observan cómo los atributos cambian conforme invocan los métodos.

Mi propuesta

Creo que ya es un buen momento para abandonar definitivamente `alert()` como mecanismo para mostrar resultados.

Mantendría únicamente los `prompt()` para ingresar datos (porque todavía no vimos formularios HTML) y haría que **todos los mensajes y el estado del objeto se muestren en la página**, simulando la salida de una consola. El alumno verá inmediatamente qué método se ejecutó, qué atributos cambiaron y qué excepción se produjo, sin necesidad de abrir las herramientas del navegador. Además, esta estructura facilitará la transición a formularios HTML en la siguiente etapa del proyecto.



perfecto, vayamos con esta sugerencia y devolveme los archivos completos actualizados

La aplicación queda formada por tres archivos. Las validaciones se centralizan en `Cliente.js`, evitando repetirlas en `App.js`.

```
fintech-cliente/  
├─ index.html  
├─ Cliente.js  
└─ App.js
```

1. Cliente.js

<> JavaScript

```
class Cliente {  
  
    // =====  
    // ATRIBUTOS PRIVADOS  
    // =====  
  
    #id;  
    #nombre;  
    #apellido;  
    #dni;  
    #email;  
    #password;  
  
    // =====  
    // CONSTRUCTOR  
    // =====  
}
```

```
constructor(id, nombre, apellido, dni, email, password) {
    this.setId(id);
    this.setNombre(nombre);
    this.setApellido(apellido);
    this.setDni(dni);
    this.setEmail(email);
    this.setPassword(password);
}

// =====
// MÉTODOS ESTÁTICOS DE VALIDACIÓN
// =====

/*
 * Los setters y la clase App utilizan estos métodos.
 * De este modo, las validaciones no se repiten.
 */

static validarId(id) {
    const idNumerico = Number(id);

    if (!Number.isInteger(idNumerico) || idNumerico <= 0) {
        throw new Error(
            "El ID debe ser un número entero positivo."
        );
    }

    return idNumerico;
}

static validarNombre(nombre) {
    if (typeof nombre !== "string") {
        throw new Error(
            "El nombre debe ser una cadena de texto."
        );
    }

    const nombreNormalizado = nombre.trim();

    if (nombreNormalizado === "") {
        throw new Error(
            "El nombre es obligatorio."
        );
    }

    if (nombreNormalizado.length < 2) {
        throw new Error(
            "El nombre debe tener al menos 2 caracteres."
        );
    }
}
```

```
        return nombreNormalizado;
    }

    static validarApellido(apellido) {
        if (typeof apellido !== "string") {
            throw new Error(
                "El apellido debe ser una cadena de texto."
            );
        }

        const apellidoNormalizado = apellido.trim();

        if (apellidoNormalizado === "") {
            throw new Error(
                "El apellido es obligatorio."
            );
        }

        if (apellidoNormalizado.length < 2) {
            throw new Error(
                "El apellido debe tener al menos 2 caracteres."
            );
        }

        return apellidoNormalizado;
    }

    static validarDni(dni) {
        const dniNormalizado = String(dni).trim();
        const expresionDni = /^d{7,8}$/;

        if (!expresionDni.test(dniNormalizado)) {
            throw new Error(
                "El DNI debe contener 7 u 8 dígitos."
            );
        }

        return dniNormalizado;
    }

    static validarEmail(email) {
        if (typeof email !== "string") {
            throw new Error(
                "El correo electrónico debe ser una cadena de texto."
            );
        }

        const emailNormalizado = email.trim().toLowerCase();
        const expresionEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

        if (!expresionEmail.test(emailNormalizado)) {
```

```
        throw new Error(
            "El correo electrónico no tiene un formato válido."
        );
    }

    return emailNormalizado;
}

static validarPassword(password) {
    if (typeof password !== "string") {
        throw new Error(
            "La contraseña debe ser una cadena de texto."
        );
    }

    if (password.length < 8) {
        throw new Error(
            "La contraseña debe tener al menos 8 caracteres."
        );
    }

    return password;
}

// =====
// GETTERS
// =====

getId() {
    return this.#id;
}

getNombre() {
    return this.#nombre;
}

getApellido() {
    return this.#apellido;
}

getDni() {
    return this.#dni;
}

getEmail() {
    return this.#email;
}

getPassword() {
    return this.#password;
}
```



```
// =====  
// SETTERS  
// =====  
  
setId(id) {  
    this.#id = Cliente.validarId(id);  
}  
  
setNombre(nombre) {  
    this.#nombre = Cliente.validarNombre(nombre);  
}  
  
setApellido(apellido) {  
    this.#apellido = Cliente.validarApellido(apellido);  
}  
  
setDni(dni) {  
    this.#dni = Cliente.validarDni(dni);  
}  
  
setEmail(email) {  
    this.#email = Cliente.validarEmail(email);  
}  
  
setPassword(password) {  
    this.#password = Cliente.validarPassword(password);  
}  
  
// =====  
// MÉTODOS DE COMPORTAMIENTO  
// =====  
  
obtenerNombreCompleto() {  
    return `${this.#nombre} ${this.#apellido}`;  
}  
  
autenticar(email, password) {  
    const emailNormalizado =  
        Cliente.validarEmail(email);  
  
    if (typeof password !== "string") {  
        throw new Error(  
            "Debe ingresar una contraseña."  
        );  
    }  
  
    return (  
        this.#email === emailNormalizado &&  
        this.#password === password  
    );  
}
```

```
}

actualizarEmail(nuevoEmail) {
  const emailNormalizado =
    Cliente.validarEmail(nuevoEmail);

  if (emailNormalizado === this.#email) {
    throw new Error(
      "El nuevo correo debe ser diferente del correo actual."
    );
  }

  this.#email = emailNormalizado;

  return "Correo electrónico actualizado correctamente.";
}

cambiarPassword(passwordActual, passwordNueva) {
  if (typeof passwordActual !== "string") {
    throw new Error(
      "Debe ingresar la contraseña actual."
    );
  }

  if (passwordActual !== this.#password) {
    throw new Error(
      "La contraseña actual es incorrecta."
    );
  }

  const passwordNuevaValidada =
    Cliente.validarPassword(passwordNueva);

  if (passwordNuevaValidada === passwordActual) {
    throw new Error(
      "La nueva contraseña debe ser diferente de la actual."
    );
  }

  this.#password = passwordNuevaValidada;

  return "Contraseña modificada correctamente.";
}

obtenerDatos() {
  return {
    id: this.#id,
    nombre: this.#nombre,
    apellido: this.#apellido,
    dni: this.#dni,
    email: this.#email
  }
}
```

```

    };
}

toString() {
    return (
        `ID: ${this.#id}\n` +
        `Nombre: ${this.#nombre}\n` +
        `Apellido: ${this.#apellido}\n` +
        `DNI: ${this.#dni}\n` +
        `Email: ${this.#email}`
    );
}
}

```

2. App.js

</> JavaScript

```

class App {

    constructor() {
        this.cliente = null;

        this.salida =
            document.getElementById("salida");

        this.estadoCliente =
            document.getElementById("estado-cliente");

        this.botonIniciar =
            document.getElementById("btn-iniciar");

        this.botonMenu =
            document.getElementById("btn-menu");

        this.botonLimpiar =
            document.getElementById("btn-limpiar");
    }

    // =====
    // INICIO DE LA APLICACIÓN
    // =====

    iniciar() {
        this.configurarEventos();

        this.mostrarMensaje(
            "Aplicación preparada. Presione «Registrar cliente» para comenzar.",
            "informacion"
        );
    }
}

```

```
configurarEventos() {  
  this.botonIniciar.addEventListener(  
    "click",  
    () => this.registrarCliente()  
  );  
  
  this.botonMenu.addEventListener(  
    "click",  
    () => this.mostrarMenu()  
  );  
  
  this.botonLimpiar.addEventListener(  
    "click",  
    () => this.limpiarHistorial()  
  );  
}  
  
// =====  
// REGISTRO DEL CLIENTE  
// =====  
  
registrarCliente() {  
  try {  
    this.mostrarTitulo("REGISTRO DE CLIENTE");  
  
    /*  
     * Cada dato se solicita por separado.  
     * Cuando el dato es inválido, se vuelve a pedir  
     * únicamente ese dato.  
     */  
  
    const id = this.solicitarDato(  
      "Ingrese el identificador del cliente:",  
      Cliente.validarId,  
      "ID"  
    );  
  
    const nombre = this.solicitarDato(  
      "Ingrese el nombre del cliente:",  
      Cliente.validarNombre,  
      "Nombre"  
    );  
  
    const apellido = this.solicitarDato(  
      "Ingrese el apellido del cliente:",  
      Cliente.validarApellido,  
      "Apellido"  
    );  
  
    const dni = this.solicitarDato(  

```

```
        "Ingrese el DNI del cliente:",
        Cliente.validarDni,
        "DNI"
    );

    const email = this.solicitarDato(
        "Ingrese el correo electrónico:",
        Cliente.validarEmail,
        "Correo electrónico"
    );

    const password = this.solicitarDato(
        "Ingrese una contraseña de al menos 8 caracteres:",
        Cliente.validarPassword,
        "Contraseña",
        true
    );

    this.cliente = new Cliente(
        id,
        nombre,
        apellido,
        dni,
        email,
        password
    );

    this.mostrarMensaje(
        "Cliente creado correctamente.",
        "exito"
    );

    this.actualizarEstadoCliente();

    this.botonMenu.disabled = false;

    this.mostrarMensaje(
        "Ahora puede utilizar el menú de operaciones.",
        "informacion"
    );

    } catch (error) {
        this.mostrarMensaje(
            error.message,
            "advertencia"
        );
    }
}

solicitarDato(
    mensaje,
```

```
        funcionValidacion,
        nombreCampo,
        ocultarValor = false
    ) {
        while (true) {
            const valorIngresado = prompt(mensaje);

            if (valorIngresado === null) {
                throw new Error(
                    "El registro del cliente fue cancelado."
                );
            }

            try {
                const valorValidado =
                    funcionValidacion(valorIngresado);

                const valorMostrado = ocultarValor
                    ? "*****"
                    : valorValidado;

                this.mostrarMensaje(
                    `${nombreCampo}: ${valorMostrado}`,
                    "dato"
                );

                return valorValidado;

            } catch (error) {
                this.mostrarMensaje(
                    `${nombreCampo}: ${error.message}`,
                    "error"
                );

                alert(
                    `${error.message}\n\nVuelva a ingresar el dato.`
                );
            }
        }
    }

    // =====
    // MENÚ DE OPERACIONES
    // =====

    mostrarMenu() {
        if (!this.cliente) {
            this.mostrarMensaje(
                "Primero debe registrar un cliente.",
                "advertencia"
            );
        }
    }
}
```

```
        return;
    }

    let opcion;

    do {
        opcion = prompt(
            "Seleccione una opción:\n\n" +
            "1. Mostrar datos del cliente\n" +
            "2. Mostrar nombre completo\n" +
            "3. Iniciar sesión\n" +
            "4. Modificar nombre\n" +
            "5. Modificar apellido\n" +
            "6. Actualizar correo electrónico\n" +
            "7. Cambiar contraseña\n" +
            "0. Cerrar menú"
        );

        if (opcion === null) {
            opcion = "0";
        }

        switch (opcion) {
            case "1":
                this.mostrarDatos();
                break;

            case "2":
                this.mostrarNombreCompleto();
                break;

            case "3":
                this.iniciarSesion();
                break;

            case "4":
                this.modificarNombre();
                break;

            case "5":
                this.modificarApellido();
                break;

            case "6":
                this.modificarEmail();
                break;

            case "7":
                this.modificarPassword();
                break;
        }
    } while (opcion !== "0");
}
```

```
        case "0":
            this.mostrarMensaje(
                "Menú cerrado.",
                "informacion"
            );
            break;

        default:
            this.mostrarMensaje(
                `La opción «${opcion}» no es válida.`,
                "error"
            );

            alert(
                "La opción ingresada no es válida."
            );
    }

    } while (opcion !== "0");
}

// =====
// OPERACIONES SOBRE EL CLIENTE
// =====

mostrarDatos() {
    this.mostrarTitulo("DATOS DEL CLIENTE");

    const datos = this.cliente.obtenerDatos();

    this.mostrarMensaje(
        `ID: ${datos.id}`,
        "dato"
    );

    this.mostrarMensaje(
        `Nombre: ${datos.nombre}`,
        "dato"
    );

    this.mostrarMensaje(
        `Apellido: ${datos.apellido}`,
        "dato"
    );

    this.mostrarMensaje(
        `DNI: ${datos.dni}`,
        "dato"
    );
}
```



```
this.mostrarMensaje(  
  `Correo electrónico: ${datos.email}`,  
  "dato"  
);  
}  
  
mostrarNombreCompleto() {  
  this.mostrarMensaje(  
    `Nombre completo: ${this.cliente.obtenerNombreCompleto()}`,  
    "exito"  
  );  
}  
  
iniciarSesion() {  
  const email = this.solicitarDato(  
    "Ingrese su correo electrónico:",  
    Cliente.validarEmail,  
    "Correo ingresado"  
  );  
  
  const password = prompt(  
    "Ingrese su contraseña:"  
  );  
  
  if (password === null) {  
    this.mostrarMensaje(  
      "Inicio de sesión cancelado.",  
      "advertencia"  
    );  
  
    return;  
  }  
  
  const autenticado = this.cliente.autenticar(  
    email,  
    password  
  );  
  
  if (autenticado) {  
    this.mostrarMensaje(  
      `Inicio de sesión correcto. Bienvenido, ${this.cliente.obtenerNombreCompleto()}`,  
      "exito"  
    );  
  } else {  
    this.mostrarMensaje(  
      "Correo electrónico o contraseña incorrectos.",  
      "error"  
    );  
  }  
}
```

```
modificarNombre() {
  const nuevoNombre = this.solicitarDato(
    "Ingrese el nuevo nombre:",
    Cliente.validarNombre,
    "Nuevo nombre"
  );

  this.cliente.setNombre(nuevoNombre);

  this.mostrarMensaje(
    "Nombre actualizado correctamente.",
    "exito"
  );

  this.actualizarEstadoCliente();
}

modificarApellido() {
  const nuevoApellido = this.solicitarDato(
    "Ingrese el nuevo apellido:",
    Cliente.validarApellido,
    "Nuevo apellido"
  );

  this.cliente.setApellido(nuevoApellido);

  this.mostrarMensaje(
    "Apellido actualizado correctamente.",
    "exito"
  );

  this.actualizarEstadoCliente();
}

modificarEmail() {
  while (true) {
    try {
      const nuevoEmail = prompt(
        "Ingrese el nuevo correo electrónico:"
      );

      if (nuevoEmail === null) {
        this.mostrarMensaje(
          "Actualización de correo cancelada.",
          "advertencia"
        );

        return;
      }
    }

    const resultado =
```

```
        this.cliente.actualizarEmail(nuevoEmail);

        this.mostrarMensaje(
            resultado,
            "exito"
        );

        this.mostrarMensaje(
            `Nuevo correo: ${this.cliente.getEmail()}`,
            "dato"
        );

        this.actualizarEstadoCliente();

        return;
    } catch (error) {
        this.mostrarMensaje(
            error.message,
            "error"
        );

        alert(
            `${error.message}\n\nVuelva a ingresar el correo.`
        );
    }
}

}

modificarPassword() {
    while (true) {
        try {
            const passwordActual = prompt(
                "Ingrese la contraseña actual:"
            );

            if (passwordActual === null) {
                this.mostrarMensaje(
                    "Cambio de contraseña cancelado.",
                    "advertencia"
                );

                return;
            }

            const passwordNueva = prompt(
                "Ingrese la nueva contraseña:"
            );

            if (passwordNueva === null) {
                this.mostrarMensaje(
```

```
        "Cambio de contraseña cancelado.",
        "advertencia"
    );

    return;
}

const resultado =
    this.cliente.cambiarPassword(
        passwordActual,
        passwordNueva
    );

this.mostrarMensaje(
    resultado,
    "exito"
);

this.mostrarMensaje(
    "Nueva contraseña: *****",
    "dato"
);

return;

} catch (error) {
    this.mostrarMensaje(
        error.message,
        "error"
    );

    alert(
        `${error.message}\n\nVuelva a ingresar los datos.`
    );
}

}

// =====
// PRESENTACIÓN EN PANTALLA
// =====

mostrarTitulo(texto) {
    const titulo = document.createElement("div");

    titulo.className = "mensaje titulo-terminal";
    titulo.textContent = `=== ${texto} ===`;

    this.salida.appendChild(titulo);
    this.desplazarSalida();
}
```

```
mostrarMensaje(mensaje, tipo = "informacion") {
    const elemento = document.createElement("div");

    elemento.className = `mensaje ${tipo}`;
    elemento.textContent = mensaje;

    this.salida.appendChild(elemento);
    this.desplazarSalida();

    console.log(mensaje);
}

actualizarEstadoCliente() {
    if (!this.cliente) {
        this.estadoCliente.innerHTML = `
            <p class="sin-cliente">
                Todavía no se registró ningún cliente.
            </p>
        `;

        return;
    }

    const datos = this.cliente.obtenerDatos();

    /*
     * Utilizamos textContent en los valores para evitar
     * insertar directamente contenido ingresado por el usuario.
     */

    this.estadoCliente.innerHTML = "";

    const campos = [
        ["ID", datos.id],
        ["Nombre", datos.nombre],
        ["Apellido", datos.apellido],
        ["DNI", datos.dni],
        ["Correo electrónico", datos.email],
        ["Contraseña", "*****"]
    ];

    campos.forEach(([etiqueta, valor]) => {
        const fila = document.createElement("div");
        fila.className = "fila-dato";

        const nombreCampo = document.createElement("span");
        nombreCampo.className = "etiqueta";
        nombreCampo.textContent = etiqueta;

        const valorCampo = document.createElement("span");
```

```

        valorCampo.className = "valor";
        valorCampo.textContent = valor;

        fila.appendChild(nombreCampo);
        fila.appendChild(valorCampo);

        this.estadoCliente.appendChild(fila);
    });
}

limpiarHistorial() {
    this.salida.innerHTML = "";

    this.mostrarMensaje(
        "Historial de operaciones limpiado.",
        "informacion"
    );
}

desplazarSalida() {
    this.salida.scrollTop =
        this.salida.scrollHeight;
}
}

// =====
// EJECUCIÓN PRINCIPAL
// =====

document.addEventListener("DOMContentLoaded", () => {
    const app = new App();
    app.iniciar();
});

```

3. index.html



```

<!DOCTYPE html>
<html lang="es">

<head>

    <meta charset="UTF-8">

    <meta
        name="viewport"
        content="width=device-width, initial-scale=1.0"
    >

```

```
<title>Sistema de Gestión Fintech</title>
```

```
<style>
```

```
/* =====
```

```
CONFIGURACIÓN GENERAL
```

Comportamiento actualizado

Al presionar **Registrar cliente**, cada valor se solicita individualmente. Por ejemplo, si se ingresa un DNI de tres dígitos:

123

se mostrará en la página:

DNI: El DNI debe contener 7 u 8 dígitos.

Después, la aplicación volverá a solicitar solamente el DNI, sin perder el ID, nombre y apellido ingresados anteriormente.

Los datos aceptados se agregan progresivamente al historial:

=== REGISTRO DE CLIENTE ===

ID: 1

Nombre: Juan

Apellido: Pérez

DNI: 32145678

Correo electrónico: juan@gmail.com

Contraseña: *****

Cliente creado correctamente.

La contraseña se oculta tanto en el historial como en el panel del cliente. Una vez creado el objeto, el botón **Abrir menú de operaciones** queda habilitado.



Vamos a crear pruebas unitarias para verificar que la creación y modificación de clientes funcionen correctamente. Utilizaremos el framework de JS para la generación y testeo. Devolveremos el paso a paso para lograrlo desde VSCode

Utilizaremos **Jest**, un framework de pruebas para JavaScript que se instala mediante npm. Las pruebas se ejecutarán con Node.js desde la terminal integrada de VS Code, sin necesidad de abrir `index.html`. La instalación oficial se realiza con `npm install --save-dev jest`. 

La estructura final será:

```
fintech-cliente/  
├─ index.html  
├─ Cliente.js  
├─ App.js  
├─ Cliente.test.js  
├─ package.json  
└─ node_modules/
```

`node_modules` se crea automáticamente al instalar Jest.

1. Preparar `Cliente.js` para las pruebas

La clase seguirá funcionando normalmente desde el navegador, pero agregaremos al final una exportación compatible con Node.js y Jest.

`Cliente.js` completo

<> JavaScript

```
class Cliente {  
  
    // =====  
    // ATRIBUTOS PRIVADOS  
    // =====  
  
    #id;  
    #nombre;  
    #apellido;  
    #dni;  
    #email;  
    #password;  
  
    // =====  
    // CONSTRUCTOR  
    // =====  
  
    constructor(id, nombre, apellido, dni, email, password) {  
        this.setId(id);  
        this.setNombre(nombre);  
        this.setApellido(apellido);  
        this.setDni(dni);  
        this.setEmail(email);  
    }  
}
```



```
        this.setPassword(password);
    }

    // =====
    // MÉTODOS ESTÁTICOS DE VALIDACIÓN
    // =====

    static validarId(id) {
        const idNumerico = Number(id);

        if (!Number.isInteger(idNumerico) || idNumerico <= 0) {
            throw new Error(
                "El ID debe ser un número entero positivo."
            );
        }

        return idNumerico;
    }

    static validarNombre(nombre) {
        if (typeof nombre !== "string") {
            throw new Error(
                "El nombre debe ser una cadena de texto."
            );
        }

        const nombreNormalizado = nombre.trim();

        if (nombreNormalizado === "") {
            throw new Error(
                "El nombre es obligatorio."
            );
        }

        if (nombreNormalizado.length < 2) {
            throw new Error(
                "El nombre debe tener al menos 2 caracteres."
            );
        }

        return nombreNormalizado;
    }

    static validarApellido(apellido) {
        if (typeof apellido !== "string") {
            throw new Error(
                "El apellido debe ser una cadena de texto."
            );
        }

        const apellidoNormalizado = apellido.trim();
```

```
    if (apellidoNormalizado === "") {
        throw new Error(
            "El apellido es obligatorio."
        );
    }

    if (apellidoNormalizado.length < 2) {
        throw new Error(
            "El apellido debe tener al menos 2 caracteres."
        );
    }

    return apellidoNormalizado;
}

static validarDni(dni) {
    const dniNormalizado = String(dni).trim();
    const expresionDni = /^\\d{7,8}$/;

    if (!expresionDni.test(dniNormalizado)) {
        throw new Error(
            "El DNI debe contener 7 u 8 dígitos."
        );
    }

    return dniNormalizado;
}

static validarEmail(email) {
    if (typeof email !== "string") {
        throw new Error(
            "El correo electrónico debe ser una cadena de texto."
        );
    }

    const emailNormalizado = email.trim().toLowerCase();
    const expresionEmail = /^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/;

    if (!expresionEmail.test(emailNormalizado)) {
        throw new Error(
            "El correo electrónico no tiene un formato válido."
        );
    }

    return emailNormalizado;
}

static validarPassword(password) {
    if (typeof password !== "string") {
        throw new Error(

```

```
        "La contraseña debe ser una cadena de texto."
    );
}

if (password.length < 8) {
    throw new Error(
        "La contraseña debe tener al menos 8 caracteres."
    );
}

return password;
}

// =====
// GETTERS
// =====

getId() {
    return this.#id;
}

getNombre() {
    return this.#nombre;
}

getApellido() {
    return this.#apellido;
}

getDni() {
    return this.#dni;
}

getEmail() {
    return this.#email;
}

getPassword() {
    return this.#password;
}

// =====
// SETTERS
// =====

setId(id) {
    this.#id = Cliente.validarId(id);
}

setNombre(nombre) {
    this.#nombre = Cliente.validarNombre(nombre);
}
```

```
}

setApellido(apellido) {
    this.#apellido = Cliente.validarApellido(apellido);
}

setDni(dni) {
    this.#dni = Cliente.validarDni(dni);
}

setEmail(email) {
    this.#email = Cliente.validarEmail(email);
}

setPassword(password) {
    this.#password = Cliente.validarPassword(password);
}

// =====
// MÉTODOS DE COMPORTAMIENTO
// =====

obtenerNombreCompleto() {
    return `${this.#nombre} ${this.#apellido}`;
}

autenticar(email, password) {
    const emailNormalizado =
        Cliente.validarEmail(email);

    if (typeof password !== "string") {
        throw new Error(
            "Debe ingresar una contraseña."
        );
    }

    return (
        this.#email === emailNormalizado &&
        this.#password === password
    );
}

actualizarEmail(nuevoEmail) {
    const emailNormalizado =
        Cliente.validarEmail(nuevoEmail);

    if (emailNormalizado === this.#email) {
        throw new Error(
            "El nuevo correo debe ser diferente del correo actual."
        );
    }
}
```

```
        this.#email = emailNormalizado;

        return "Correo electrónico actualizado correctamente.";
    }

    cambiarPassword(passwordActual, passwordNueva) {
        if (typeof passwordActual !== "string") {
            throw new Error(
                "Debe ingresar la contraseña actual."
            );
        }

        if (passwordActual !== this.#password) {
            throw new Error(
                "La contraseña actual es incorrecta."
            );
        }

        const passwordNuevaValidada =
            Cliente.validarPassword(passwordNueva);

        if (passwordNuevaValidada === passwordActual) {
            throw new Error(
                "La nueva contraseña debe ser diferente de la actual."
            );
        }

        this.#password = passwordNuevaValidada;

        return "Contraseña modificada correctamente.";
    }

    obtenerDatos() {
        return {
            id: this.#id,
            nombre: this.#nombre,
            apellido: this.#apellido,
            dni: this.#dni,
            email: this.#email
        };
    }

    toString() {
        return (
            `ID: ${this.#id}\n` +
            `Nombre: ${this.#nombre}\n` +
            `Apellido: ${this.#apellido}\n` +
            `DNI: ${this.#dni}\n` +
            `Email: ${this.#email}`
        );
    }
}
```

```
    }  
  }  
  
  /*  
   * Esta condición permite exportar Cliente cuando el archivo  
   * se ejecuta mediante Node.js y Jest.  
   *  
   * En el navegador, "module" no existe, por lo que esta parte  
   * simplemente se ignora.  
   */  
  
  if (typeof module !== "undefined" && module.exports) {  
    module.exports = Cliente;  
  }
```

El único agregado respecto de la aplicación anterior es:

JavaScript

```
if (typeof module !== "undefined" && module.exports) {  
  module.exports = Cliente;  
}
```

Esto permite usar la misma clase:

- desde `index.html`, como una clase global;
- desde Jest, mediante `require()`.

2. Crear el archivo de pruebas

En la carpeta principal del proyecto, crear:

```
Cliente.test.js
```

Jest reconoce automáticamente archivos terminados en:

```
.test.js
```

Cliente.test.js completo

JavaScript

```
const Cliente = require("./Cliente");  
  
describe("Pruebas unitarias de la clase Cliente", () => {  
  
  /*  
   * Esta variable estará disponible para todas  
   * las pruebas del bloque describe.  
   */  
  
  // ...  
});
```

```
*/

let cliente;

/*
 * beforeEach se ejecuta antes de cada prueba.
 *
 * De este modo, cada test comienza con un objeto nuevo
 * y no depende de modificaciones realizadas por otros tests.
 */

beforeEach(() => {
  cliente = new Cliente(
    1,
    "Juan",
    "Pérez",
    "32145678",
    "juan@gmail.com",
    "clave123"
  );
});

// =====
// PRUEBAS DE CREACIÓN
// =====

describe("Creación de clientes", () => {

  test("debe crear un cliente con datos válidos", () => {
    expect(cliente).toBeInstanceOf(Cliente);
  });

  test("debe guardar correctamente los datos del cliente", () => {
    expect(cliente.getId()).toBe(1);
    expect(cliente.getNombre()).toBe("Juan");
    expect(cliente.getApellido()).toBe("Pérez");
    expect(cliente.getDni()).toBe("32145678");
    expect(cliente.getEmail()).toBe("juan@gmail.com");
  });

  test("debe devolver el nombre completo", () => {
    expect(
      cliente.obtenerNombreCompleto()
    ).toBe("Juan Pérez");
  });

  test("debe normalizar el correo electrónico", () => {
    const otroCliente = new Cliente(
      2,
      "Ana",
      "Gómez",

```

```
        "30123456",
        "  ANA.GOMEZ@GMAIL.COM  ",
        "password123"
    );

    expect(
        otroCliente.getEmail()
    ).toBe("ana.gomez@gmail.com");
});

test("debe eliminar espacios del nombre y apellido", () => {
    const otroCliente = new Cliente(
        3,
        "  Pedro  ",
        "  López  ",
        "29123456",
        "pedro@gmail.com",
        "password123"
    );

    expect(
        otroCliente.getNombre()
    ).toBe("Pedro");

    expect(
        otroCliente.getApellido()
    ).toBe("López");
});

});

// =====
// PRUEBAS DE MODIFICACIÓN
// =====

describe("Modificación de clientes", () => {

    test("debe modificar el nombre", () => {
        cliente.setNombre("Juan Carlos");

        expect(
            cliente.getNombre()
        ).toBe("Juan Carlos");
    });

    test("debe modificar el apellido", () => {
        cliente.setApellido("González");

        expect(
            cliente.getApellido()
        ).toBe("González");
    });
});
```



```
});

test("debe modificar el DNI", () => {
  cliente.setDni("30111222");

  expect(
    cliente.getDni()
  ).toBe("30111222");
});

test("debe actualizar el correo electrónico", () => {
  const resultado = cliente.actualizarEmail(
    "juan.perez@gmail.com"
  );

  expect(
    cliente.getEmail()
  ).toBe("juan.perez@gmail.com");

  expect(resultado).toBe(
    "Correo electrónico actualizado correctamente."
  );
});

test("debe cambiar la contraseña", () => {
  const resultado = cliente.cambiarPassword(
    "clave123",
    "NuevaClave2026"
  );

  expect(
    cliente.getPassword()
  ).toBe("NuevaClave2026");

  expect(resultado).toBe(
    "Contraseña modificada correctamente."
  );
});

test("debe autenticar con los nuevos datos", () => {
  cliente.actualizarEmail(
    "juan.perez@gmail.com"
  );

  cliente.cambiarPassword(
    "clave123",
    "NuevaClave2026"
  );

  const autenticado = cliente.autenticar(
    "juan.perez@gmail.com",
```

```
        "NuevaClave2026"
    );

    expect(authenticado).toBe(true);
});

});

// =====
// PRUEBAS DE EXCEPCIONES
// =====

describe("Validaciones y excepciones", () => {

    test("no debe crear un cliente con ID negativo", () => {
        expect(() => {
            new Cliente(
                -1,
                "Ana",
                "Gómez",
                "30123456",
                "ana@gmail.com",
                "password123"
            );
        }).toThrow(
            "El ID debe ser un número entero positivo."
        );
    });

    test("no debe crear un cliente con nombre vacío", () => {
        expect(() => {
            new Cliente(
                2,
                "",
                "Gómez",
                "30123456",
                "ana@gmail.com",
                "password123"
            );
        }).toThrow(
            "El nombre es obligatorio."
        );
    });

    test("no debe aceptar un DNI inválido", () => {
        expect(() => {
            cliente.setDni("123");
        }).toThrow(
            "El DNI debe contener 7 u 8 dígitos."
        );
    });
});
```

```
test("no debe aceptar un correo electrónico inválido", () => {
  expect(() => {
    cliente.setEmail("correo-invalido");
  }).toThrow(
    "El correo electrónico no tiene un formato válido."
  );
});

test("no debe aceptar una contraseña corta", () => {
  expect(() => {
    cliente.setPassword("123");
  }).toThrow(
    "La contraseña debe tener al menos 8 caracteres."
  );
});

test("no debe cambiar la contraseña si la actual es incorrecta", () => {
  expect(() => {
    cliente.cambiarPassword(
      "claveIncorrecta",
      "NuevaClave2026"
    );
  }).toThrow(
    "La contraseña actual es incorrecta."
  );
});

test("no debe actualizar el cliente con el mismo correo", () => {
  expect(() => {
    cliente.actualizarEmail(
      "juan@gmail.com"
    );
  }).toThrow(
    "El nuevo correo debe ser diferente del correo actual."
  );
});

});

});
```

3. Abrir el proyecto en VS Code

Desde VS Code:

Archivo → Abrir carpeta

Seleccionar la carpeta:

```
fintech-cliente
```

Abrir la terminal integrada mediante:

```
Terminal → Nueva terminal
```

o con el atajo:

```
Ctrl + `
```

La terminal integrada de VS Code se abre normalmente en la raíz de la carpeta de trabajo, por lo que los comandos se ejecutarán directamente dentro del proyecto.

 Visual Studio Co... +1

4. Verificar Node.js y npm

Ejecutar:

```
</> Bash
```

```
node --version
```

Luego:

```
</> Bash
```

```
npm --version
```

Por ejemplo, la terminal podría mostrar:

```
v24.4.1
```

y:

```
11.4.2
```

No es necesario tener exactamente esas versiones. Lo importante es que ambos comandos respondan con un número de versión.

Node.js permite ejecutar JavaScript fuera del navegador y normalmente incluye npm, el gestor que utilizaremos para instalar Jest.  Node.js +1

Si los comandos no son reconocidos, será necesario instalar una versión LTS de Node.js y luego reiniciar VS Code.

5. Inicializar el proyecto con npm

En la terminal, ejecutar:

```
</> Bash
```

```
npm init -y
```

Este comando creará automáticamente:

```
package.json
```

Inicialmente tendrá una estructura similar a esta:

```
</> JSON
```

```
{
  "name": "fintech-cliente",
  "version": "1.0.0",
  "description": "",
  "main": "App.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

6. Instalar Jest

Ejecutar:

```
</> Bash
```

```
npm install --save-dev jest
```

También puede escribirse de forma abreviada:

```
</> Bash
```

```
npm install -D jest
```

Jest quedará registrado como una dependencia de desarrollo y npm creará:

```
node_modules/
package-lock.json
```

La documentación oficial de Jest recomienda instalarlo localmente en el proyecto como dependencia de desarrollo.  Jest

7. Configurar `package.json`

Modificar la sección `scripts`.

Reemplazar:

</> JSON

```
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1"  
}
```

por:

</> JSON

```
"scripts": {  
  "test": "jest",  
  "test:watch": "jest --watchAll",  
  "test:coverage": "jest --coverage"  
}
```

`package.json` completo

La versión exacta de Jest será la que npm instale en el equipo. Por eso no conviene copiar manualmente un número de versión: hay que conservar el generado por `npm install`.

El archivo quedará similar a:

</> JSON

```
{  
  "name": "fintech-cliente",  
  "version": "1.0.0",  
  "description": "Sistema de gestión de clientes para una Fintech",  
  "main": "App.js",  
  "scripts": {  
    "test": "jest",  
    "test:watch": "jest --watchAll",  
    "test:coverage": "jest --coverage"  
  },  
  "keywords": [  
    "javascript",  
    "poo",  
    "fintech",  
    "jest"  
  ],  
  "author": "",  
  "license": "ISC",  
  "devDependencies": {  
    "jest": "^30.0.0"  
  }  
}
```

```
}  
}
```

El valor real de:

```
</> JSON
```

```
"jest": "^30.0.0"
```

puede ser diferente. Debemos dejar el que haya colocado npm.

8. Ejecutar todas las pruebas

En la terminal integrada:

```
</> Bash
```

```
npm test
```

También se puede ejecutar directamente:

```
</> Bash
```

```
npx jest
```

El resultado esperado será parecido a:

```
PASS ./Cliente.test.js
```

```
Pruebas unitarias de la clase Cliente
```

```
Creación de clientes
```

- ✓ debe crear un cliente con datos válidos
- ✓ debe guardar correctamente los datos del cliente
- ✓ debe devolver el nombre completo
- ✓ debe normalizar el correo electrónico
- ✓ debe eliminar espacios del nombre y apellido

```
Modificación de clientes
```

- ✓ debe modificar el nombre
- ✓ debe modificar el apellido
- ✓ debe modificar el DNI
- ✓ debe actualizar el correo electrónico
- ✓ debe cambiar la contraseña
- ✓ debe autenticar con los nuevos datos

```
Validaciones y excepciones
```

- ✓ no debe crear un cliente con ID negativo
- ✓ no debe crear un cliente con nombre vacío
- ✓ no debe aceptar un DNI inválido
- ✓ no debe aceptar un correo electrónico inválido
- ✓ no debe aceptar una contraseña corta
- ✓ no debe cambiar la contraseña si la actual es incorrecta
- ✓ no debe actualizar el cliente con el mismo correo

Test Suites: 1 passed, 1 total
Tests: 18 passed, 18 total

9. Ejecutar las pruebas automáticamente al guardar

Usar:

```
</> Bash  
npm run test:watch
```

Jest quedará observando los archivos del proyecto.

Cuando se modifique y guarde:

```
Cliente.js
```

O:

```
Cliente.test.js
```

las pruebas volverán a ejecutarse automáticamente.

Para finalizar el modo observación:

```
Ctrl + C
```

En Windows, la terminal podría preguntar:

```
¿Desea terminar el trabajo por lotes (S/N)?
```

Responder:

```
S
```

10. Ejecutar una sola prueba

Para ejecutar solamente los tests cuyo nombre contenga "modificar":

```
</> Bash  
npx jest -t "modificar"
```

Para ejecutar únicamente el bloque relacionado con la creación:

```
</> Bash
```



```
npx jest -t "Creación de clientes"
```

Para ejecutar solamente el archivo:

```
</> Bash
```

```
npx jest Cliente.test.js
```

11. Generar un informe de cobertura

Ejecutar:

```
</> Bash
```

```
npm run test:coverage
```

Jest mostrará una tabla similar a:

File	% Stmts	% Branch	% Funcs	% Lines
All files	92.50	85.71	94.44	92.50
Cliente.js	92.50	85.71	94.44	92.50

Además, se creará la carpeta:

```
coverage/
```

Dentro de ella habrá un informe HTML:

```
coverage/lcov-report/index.html
```

Ese archivo puede abrirse en el navegador para analizar:

- instrucciones ejecutadas;
- funciones probadas;
- decisiones lógicas recorridas;
- líneas todavía no cubiertas.

12. Explicación de las principales instrucciones de Jest

```
describe()
```

Agrupar pruebas relacionadas:

```
</> JavaScript
```

```
describe("Creación de clientes", () => {  
    // Tests de creación  
});
```

Sirve para organizar las pruebas en bloques.

test()

Define una prueba unitaria:

```
<> JavaScript  
  
test("debe modificar el nombre", () => {  
    cliente.setNombre("Juan Carlos");  
  
    expect(cliente.getNombre()).toBe("Juan Carlos");  
});
```

También puede utilizarse `it()`:

```
<> JavaScript  
  
it("debe modificar el nombre", () => {  
    // Prueba  
});
```

`test()` e `it()` tienen la misma finalidad.

expect()

Indica el resultado que esperamos:

```
<> JavaScript  
  
expect(cliente.getNombre()).toBe("Juan");
```

toBe()

Comprueba igualdad estricta en valores simples:

```
<> JavaScript  
  
expect(cliente.getId()).toBe(1);
```

Es equivalente conceptualmente a verificar:

```
<> JavaScript  
  
cliente.getId() === 1
```

toBeInstanceOf()

Comprueba que el objeto pertenezca a una clase:

</> JavaScript

```
expect(cliente).toBeInstanceOf(Cliente);
```

toThrow()

Comprueba que una operación lance una excepción:

</> JavaScript

```
expect(() => {
  cliente.setDni("123");
}).toThrow(
  "El DNI debe contener 7 u 8 dígitos."
);
```

La operación debe colocarse dentro de una función:

</> JavaScript

```
() => {
  cliente.setDni("123");
}
```

No debemos escribir:

</> JavaScript

```
expect(cliente.setDni("123")).toThrow();
```

porque en ese caso la excepción se produciría antes de que Jest pudiera verificarla.

beforeEach()

Se ejecuta antes de cada prueba:

</> JavaScript

```
beforeEach(() => {
  cliente = new Cliente(
    1,
    "Juan",
    "Pérez",
    "32145678",
    "juan@gmail.com",
    "clave123"
  );
});
```

Esto garantiza que cada test trabaje con un cliente nuevo e independiente.

13. Comprobar que una prueba realmente puede fallar

Para mostrar a los alumnos cómo funciona Jest, se puede cambiar temporalmente esta prueba:

```
</> JavaScript
```

```
expect(cliente.getNombre()).toBe("Juan");
```

por:

```
</> JavaScript
```

```
expect(cliente.getNombre()).toBe("Pedro");
```

Al ejecutar:

```
</> Bash
```

```
npm test
```

Jest informará:

```
Expected: "Pedro"
```

```
Received: "Juan"
```

Luego se debe restaurar el valor correcto:

```
</> JavaScript
```

```
expect(cliente.getNombre()).toBe("Juan");
```

Una prueba útil no solamente debe aprobar cuando el código funciona: también debe fallar cuando el comportamiento es incorrecto.

14. Diferencia entre la aplicación y las pruebas

La aplicación continúa ejecutándose desde:

```
index.html
```

y utiliza:

```
App.js
```

```
Cliente.js
```

Las pruebas se ejecutan desde la terminal:

</> Bash

npm test

y utilizan:

Cliente.test.js

Cliente.js

`App.js` no se prueba en esta etapa porque contiene interacción con elementos HTML, botones, `prompt()` y el DOM. Por ahora estamos realizando pruebas unitarias del modelo `Cliente`, que contiene la lógica de creación, modificación, autenticación y validación.

Para probar posteriormente `App.js`, necesitaremos trabajar con un entorno DOM simulado, por ejemplo `jsdom`, o separar todavía más la lógica de presentación de la lógica del sistema.

    Fuentes